



Bases de Dados

- **Aula 12: Funções de Grupo e Cláusula GROUP BY**

Prof. Eduardo Corrêa Gonçalves

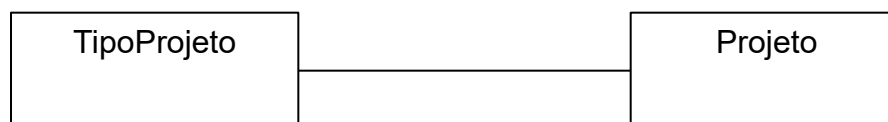
Data 04/11/2022

Tópicos

- Introdução + Base de Dados Exemplo
- Funções de Grupo
- Clausula GROUP BY
- A filosofia *Split-Apply-Combine*
- Clausula HAVING

Introdução

- A instrução SELECT oferece funções para a **produção de resultados agregados**.
- Elas são conhecidas como **funções de grupo** e serão apresentadas nesta aula com o uso de exemplos baseados nas tabelas *Projeto* e *TipoProjeto*.



TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

Introdução

- Script SQL para a criar e popular as tabelas:

```
CREATE TABLE TipoProjeto  
(idTipo CHAR(1) NOT NULL,  
 descTipo VARCHAR(30) NOT NULL,  
 PRIMARY KEY(idTipo));
```

```
CREATE TABLE Projeto  
(codigo CHAR(2) NOT NULL,  
 tipo CHAR(1) NOT NULL,  
 local CHAR(2) NOT NULL,  
 custo NUM,  
 PRIMARY KEY(codigo),  
 FOREIGN KEY (tipo) REFERENCES TipoProjeto(idTipo));
```

```
INSERT INTO TipoProjeto VALUES('A', 'Reforma Estrutural');  
INSERT INTO TipoProjeto VALUES('B', 'Manutenção de Fachada');
```

```
INSERT INTO Projeto VALUES('P1', 'A', 'RJ', 500000);  
INSERT INTO Projeto VALUES('P2', 'A', 'DF', 900000);  
INSERT INTO Projeto VALUES('P3', 'B', 'RJ', 150000);  
INSERT INTO Projeto VALUES('P4', 'A', 'DF', 1000000);  
INSERT INTO Projeto VALUES('P5', 'B', 'RJ', NULL);  
INSERT INTO Projeto VALUES('P6', 'A', 'SP', 850000);
```

Introdução

- A tabela *Projeto* armazena os projetos correntemente conduzidos por uma empresa de reforma de edifícios.

- Atributos:**

- codigo (codigo do projeto);
 - tipo (FK para *TipoProjeto*);
 - local (UF onde obra esta sendo realizada); e
 - custo (orcamento da obra em R\$).
- Observe que o projeto 'P5' ainda nao tem o orcamento definido (esta com valor NULL armazenado na coluna "custo").

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

Funções de Grupo: COUNT

- Funcao COUNT(*a*)
 - Determina o numero de ocorrencias de valores nao-nulos para o atributo *a* de uma tabela.

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

```
SELECT  
    COUNT(codigo),  
    COUNT(custo)  
FROM Projeto;
```

COUNT(codigo)	COUNT(custo)
6	5

Funções de Grupo: COUNT

- Funcao COUNT(*)
 - Utilizada para determinar o numero total de linhas de uma tabela.

Projeto

	codigo	tipo	local	custo
1	P1	A	RJ	500000
2	P2	A	DF	900000
3	P3	B	RJ	150000
4	P4	A	DF	1000000
5	P5	B	RJ	NULL
6	P6	A	SP	850000

```
SELECT COUNT(*)  
FROM Projeto;
```

COUNT(*)
6

Funções de Grupo: MAX, MIN, AVG, SUM

- Funcoes MIN(*a*), MAX(*a*), AVG(*a*) e SUM(*a*)
 - MIN(*a*): retorna o valor m nimo do atributo *a* de uma tabela.
 - MAX(*a*): retorna o valor maximo do atributo *a* de uma tabela.
 - AVG(*a*): retorna o valor medio do atributo *a* de uma tabela.
 - SUM(*a*): retorna o somatorio dos valores de *a* em todas as linhas da tabela.
- **IMPORTANTE:** linhas com valores NULL para *a* sao ignoradas por estas funcoes

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

```
SELECT
    MIN(custo), MAX(custo),
    AVG(custo), SUM(custo)
FROM Projeto;
```

MIN(custo)	MAX(custo)	AVG(custo)	SUM(custo)
150000	1000000	680000	3400000

Cláusula GROUP BY

- A cláusula GROUP BY pode ser combinada com as funções de grupo para permitir a produção de **resultados agregados por uma ou mais variáveis**.
- Em outras palavras, com o GROUP BY torna-se possível **produzir tabelas** utilizando SQL.
- O exemplo a seguir obtém os valores mínimo, máximo, médio e o somatório do custo **por tipo de projeto**.

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

```

SELECT
    tipo,
    MIN(custo), MAX(custo),
    AVG(custo), SUM(custo)
FROM Projeto
GROUP BY tipo;

```

tipo	MIN(custo)	MAX(custo)	AVG(custo)	SUM(custo)
A	500000	1000000	812500	3250000
B	150000	150000	150000	150000

Cláusula GROUP BY

- Observacao importante:
- A variavel categorica pela qual os resultados serao agregados deve ser indicada na lista do SELECT e ao lado da clausula GROUP BY:

```
SELECT tipo, MIN(custo), MAX(custo), AVG(custo), SUM(custo)  
FROM Projeto  
GROUP BY tipo;
```

tipo	MIN(custo)	MAX(custo)	AVG(custo)	SUM(custo)
A	500000	1000000	812500	3250000
B	150000	150000	150000	150000

GROUP BY com mais de um atributo

- É possível compor **grupos** formados por **mais de um atributo**.
- O exemplo a seguir, produz o somatório do orçamento por tipo de projeto e local.
- Os resultados são ordenados por “tipo” e “local” (o uso de ORDER BY não é requerido – ele foi utilizando nesse exemplo apenas para melhorar a apresentação dos resultados).

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

```
SELECT tipo, local, SUM(custo)
FROM Projeto
GROUP BY tipo, local
ORDER BY tipo, local;
```

tipo	local	SUM(custo)
A	DF	1900000
A	RJ	500000
A	SP	850000
B	RJ	150000

GROUP BY com WHERE

- A clausula GROUP BY pode ser utilizada em conjunto com o WHERE.
- É importante saber que o **WHERE** é executado antes do **GROUP BY**, para definir as linhas que, de fato, serão levadas em conta para produzir a tabulação.
- Não por acaso, a posição da clausula GROUP BY em um comando SELECT é:
 - **Depois** do **FROM** e **WHERE**
 - Mas **antes** do **ORDER BY** (como mostrado no slide 11)
- No slide seguinte, analisaremos a forma pela qual o SGBD internamente executa a instrução abaixo:

```
SELECT tipo, AVG(custo)
FROM Projeto
WHERE local < > 'RJ'
GROUP BY tipo;
```

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- `SELECT tipo, AVG(custo) FROM Projeto
WHERE local < > 'RJ'
GROUP BY tipo;`

Buffer

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- **SELECT** tipo, **AVG(custo)** **FROM** Projeto
WHERE local < > 'RJ'
GROUP BY tipo;
- **PASSO 1:** execucao do **FROM**
 - Nesse caso, nao ha nenhum **JOIN**, entao *imagine* que o SGBD simplesmente faz uma copia da tabela *Projeto* em sua memoria de trabalho (*buffer*)

Buffer

RESULT_TEMP1

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

Execução Passo-a-Passo

Projeto

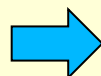
codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- **SELECT** tipo, **AVG(custo)** **FROM** Projeto
WHERE local **< >** 'RJ'
GROUP BY tipo;
- **PASSO 2:** execucao do **WHERE**
 - As linhas referentes ao local 'RJ' sao descartadas

Buffer

RESULT_TEMP1

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000



RESULT_TEMP2

codigo	tipo	local	custo
P2	A	DF	900000
P4	A	DF	1000000
P6	A	SP	850000

Execução Passo-a-Passo

Projeto

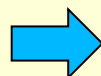
codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- **SELECT** tipo, **AVG(custo)** FROM Projeto
WHERE local < > 'RJ'
GROUP BY tipo
ORDER BY local;
- **PASSO 3:** producao da tabulacao
 - Veja que a tabulacao sera computada considerando apenas os dados filtrados pelo WHERE.

Buffer

RESULT_TEMP1

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000



RESULT_TEMP2

codigo	tipo	local	custo
P2	A	DF	900000
P4	A	DF	1000000
P6	A	SP	850000

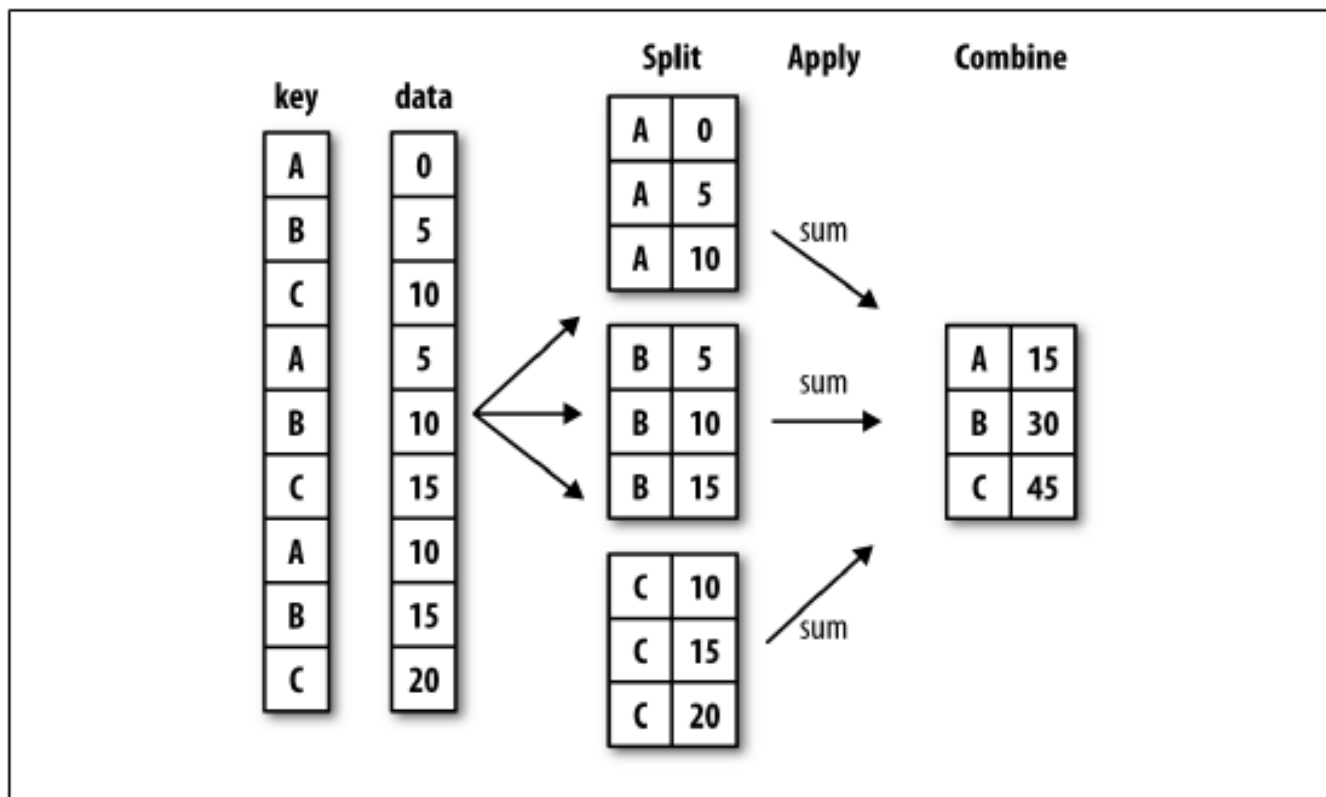


RESULTADO FINAL

tipo	AVG(custo)
A	916666.6666666666

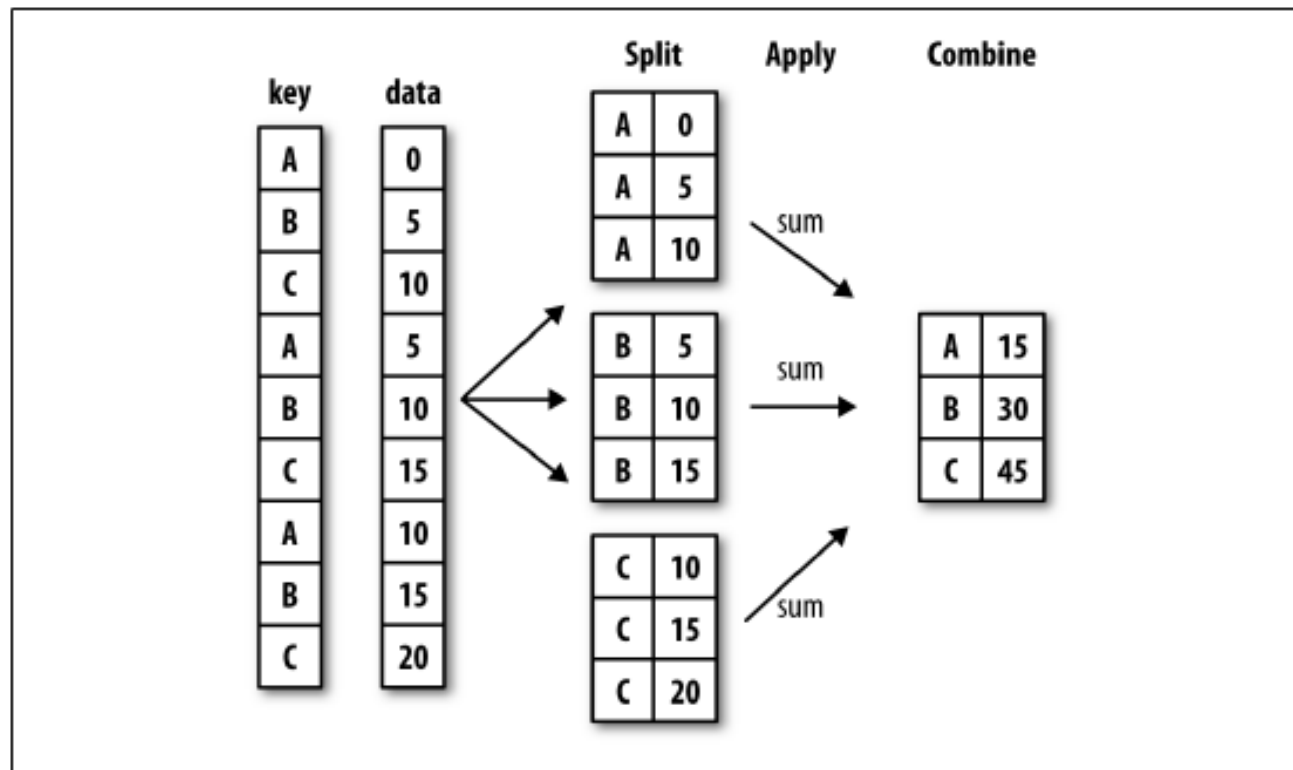
Split-Apply-Combine

- Outra explicação interessante é dada por Hadley Wickam (autor de pacotes R) .
 - Ele criou o termo **split-apply-combine** para falar de operações de agrupamento.



Split-Apply-Combine

1. **SPLIT** → Primeiro os dados são divididos pela chave fornecida no GROUP BY.
2. **APPLY** → Uma vez que isso é feito, uma função é aplicada a cada grupo, produzindo um valor agregado.
3. **COMBINE** → Por fim, os resultados são combinados para gerar a resposta final da consulta ao usuário.



GROUP BY com JOIN

- A clausula GROUP BY pode ser normalmente utilizada em instrucoes SELECT que possuam JOINS.
- Nesse caso, o fluxo de execucao sera o mesmo mostrado no exemplo anterior. Ou seja, o SGBD ira executar:
 - Primeiro o FROM (*logo, resolvera de cara os JOINS*)
 - Depois o WHERE (caso exista)
 - Depois o GROUP BY
 - E por fim o ORDER BY (caso exista)
- No slide seguinte, analisaremos a forma pela qual o SGBD internamente executa a instrucao abaixo:

```
SELECT t.descTipo, COUNT(*)  
FROM Projeto p  
INNER JOIN TipoProjeto t ON (p.tipo = t.idTipo)  
WHERE local = 'RJ'  
GROUP BY t.descTipo  
ORDER BY COUNT(*)
```

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- SELECT t.descTipo, COUNT(*)
FROM Projeto p
INNER JOIN TipoProjeto t ON (p.tipo = t.idTipo)
WHERE local = 'RJ'
GROUP BY t.descTipo
ORDER BY COUNT(*)

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

Buffer

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- **SELECT** t.descTipo, COUNT(*)
FROM Projeto p
INNER JOIN TipoProjeto t **ON** (p.tipo = t.idTipo)
WHERE local = 'RJ'
GROUP BY t.descTipo
ORDER BY COUNT(*)

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

- **PASSO 1:** FROM e resolvido, ou seja a juncao interna de *Projeto* com *TipoProjeto* e realizada.

Buffer

RESULT_TEMP1

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P2	A	DF	900000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P4	A	DF	1000000	A	Reforma Estrutural
P5	B	RJ	NULL	B	Manutenção de Fachada
P6	A	SP	850000	A	Reforma Estrutural

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- `SELECT t.descTipo, COUNT(*)`
`FROM Projeto p`
`INNER JOIN TipoProjeto t ON (p.tipo = t.idTipo)`
`WHERE local = 'RJ'`
`GROUP BY t.descTipo`
`ORDER BY COUNT(*)`

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

- **PASSO 2:** execucao do WHERE.
 - Apenas as linhas do 'RJ' sao mantidas

Buffer

RESULT_TEMP1

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P2	A	DF	900000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P4	A	DF	1000000	A	Reforma Estrutural
P5	B	RJ	NULL	B	Manutenção de Fachada
P6	A	SP	850000	A	Reforma Estrutural



RESULT_TEMP2

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P5	B	RJ	NULL	B	Manutenção de Fachada

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

- **SELECT** t.descTipo, **COUNT(*)**
FROM Projeto p
INNER JOIN TipoProjeto t ON (p.tipo = t.idTipo)
WHERE local = 'RJ'
GROUP BY t.descTipo
ORDER BY COUNT(*)
- **PASSO 3:** producao da tabulacao
 - Leva em conta apenas as linhas definidas pelo WHERE

Buffer

RESULT_TEMP1

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P2	A	DF	900000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P4	A	DF	1000000	A	Reforma Estrutural
P5	B	RJ	NULL	B	Manutenção de Fachada
P6	A	SP	850000	A	Reforma Estrutural

RESULT_TEMP2

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P5	B	RJ	NULL	B	Manutenção de Fachada

RESULT_TEMP3

descTipo	COUNT(*)
Manutenção de Fachada	2
Reforma Estrutural	1

Execução Passo-a-Passo

Projeto

codigo	tipo	local	custo
P1	A	RJ	500000
P2	A	DF	900000
P3	B	RJ	150000
P4	A	DF	1000000
P5	B	RJ	NULL
P6	A	SP	850000

- **SELECT** t.descTipo, COUNT(*)
FROM Projeto p
INNER JOIN TipoProjeto t ON (p.tipo = t.idTipo)
WHERE local = 'RJ'
GROUP BY t.descTipo
ORDER BY COUNT(*)

TipoProjeto

idTipo	descTipo
A	Reforma Estrutural
B	Manutenção de Fachada

- **PASSO 4:** ordenacao dos resultados
 - Veja que mandamos ordenar os resultados pelo valor do COUNT(*), de forma crescente

Buffer

RESULT_TEMP1

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P2	A	DF	900000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P4	A	DF	1000000	A	Reforma Estrutural
P5	B	RJ	NULL	B	Manutenção de Fachada
P6	A	SP	850000	A	Reforma Estrutural

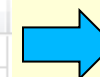
RESULT_TEMP2

codigo	tipo	local	custo	idTipo	descTipo
P1	A	RJ	500000	A	Reforma Estrutural
P3	B	RJ	150000	B	Manutenção de Fachada
P5	B	RJ	NULL	B	Manutenção de Fachada



RESULT_TEMP3

descTipo	COUNT(*)
Manutenção de Fachada	2
Reforma Estrutural	1



RESULTADO FINAL

descTipo	COUNT(*)
Reforma Estrutural	1
Manutenção de Fachada	2

Cláusula HAVING

- Caso voce queira **eliminar algum grupo do resultado final**, torna-se necessario utilizar a clausula **HAVING**.
- Veja, o exemplo a seguir, onde obtem uma tabela com a soma do custo por tipo e local do projeto

Projeto

	codigo	tipo	local	custo
1	P1	A	RJ	500000
2	P2	A	DF	900000
3	P3	B	RJ	150000
4	P4	A	DF	1000000
5	P5	B	RJ	NULL
6	P6	A	SP	850000

```
SELECT  tipo, local, SUM(custo)
FROM Projeto
GROUP BY tipo, local
ORDER BY tipo, local;
```

tipo	local	SUM(custo)
A	DF	1900000
A	RJ	500000
A	SP	850000
B	RJ	150000

Cláusula HAVING

- Utilizando a cláusula **HAVING**, podemos remover alguns resultados da tabulação final, ou seja, **dos resultados produzidos após a tabulação**.
- No exemplo abaixo, são eliminados todos os grupos em que o somatório do custo é inferior a R\$ 600.000,00 (*compare com o exemplo do slide anterior*).

Projeto

	codigo	tipo	local	custo
1	P1	A	RJ	500000
2	P2	A	DF	900000
3	P3	B	RJ	150000
4	P4	A	DF	1000000
5	P5	B	RJ	NULL
6	P6	A	SP	850000

```
SELECT tipo, local, SUM(custo)
FROM Projeto
GROUP BY tipo, local
HAVING SUM(custo) > 600000
ORDER BY tipo, local;
```

tipo	local	SUM(custo)
A	DF	1900000
A	SP	850000

Cláusula HAVING

- Não confunda **HAVING** com **WHERE** !!!
 - O **WHERE** remove linhas **antes da tabulação** ser produzida.
 - Por sua vez, a cláusula **HAVING** serve para remover linhas referentes aos resultados produzidos **após a tabulação**.

```
SELECT tipo, local, SUM(custo)
FROM Projeto
GROUP BY tipo, local
HAVING SUM(custo) > 600000
ORDER BY tipo, local;
```

Cláusula HAVING

- Observacoes importantes:
 - Normalmente, nao se usa HAVING sem GROUP BY, ja que o HAVING serve exatamente para filtrar resultados produzidos pelo GROUP BY.
 - Em um comando SELECT, a clausula HAVING deve ser colocada sempre depois do GROUP BY (mas antes do ORDER BY).
 - A clausula HAVING podera envolver qualquer atributo que faca parte da tabela de resultados final.
 - Por isso, todos os SQLs abaixo sao validos:

SELECT tipo, local, SUM(custo)	SELECT tipo, local, SUM(custo)	SELECT tipo, local, SUM(custo)
FROM Projeto	FROM Projeto	FROM Projeto
GROUP BY tipo, local	GROUP BY tipo, local	GROUP BY tipo, local
HAVING SUM(custo) > 600000	HAVING tipo = 'A'	HAVING local = 'RJ'
ORDER BY tipo, local;	ORDER BY tipo, local;	ORDER BY tipo, local;

Comentários Finais

- As funções de grupo e as cláusulas GROUP BY e HAVING podem ser normalmente utilizadas em consultas contendo junção de tabelas, contendo a cláusula WHERE, ORDER BY, etc.
- Com isso, torna-se possível produzir tabulações com SQL!